

1: Prozesse & Threads

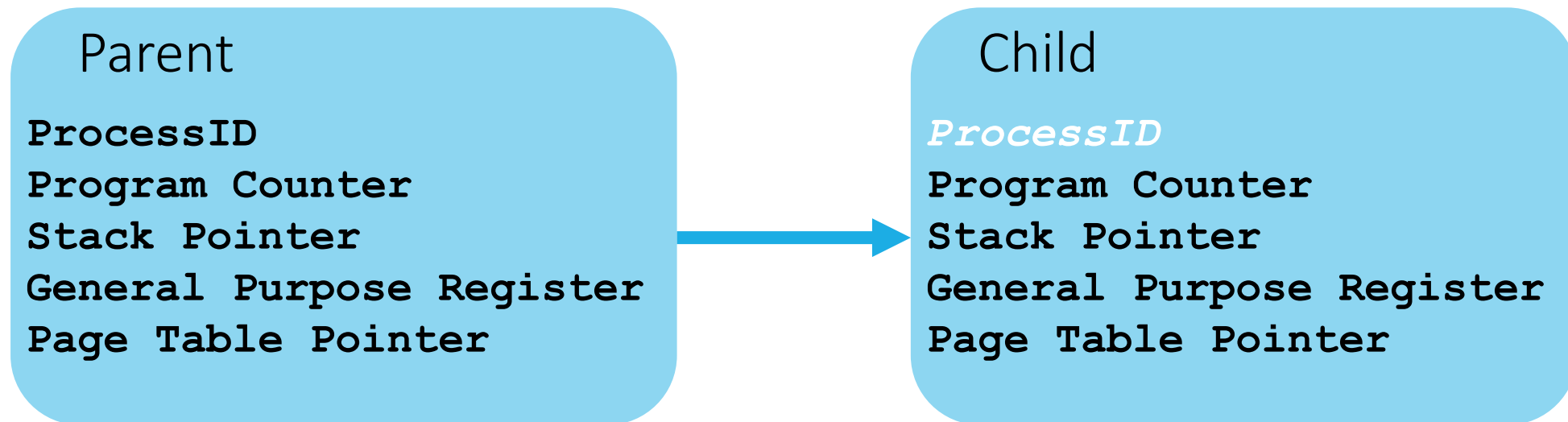
TUTORIUM 24 - BuS 2024

Frage 1:

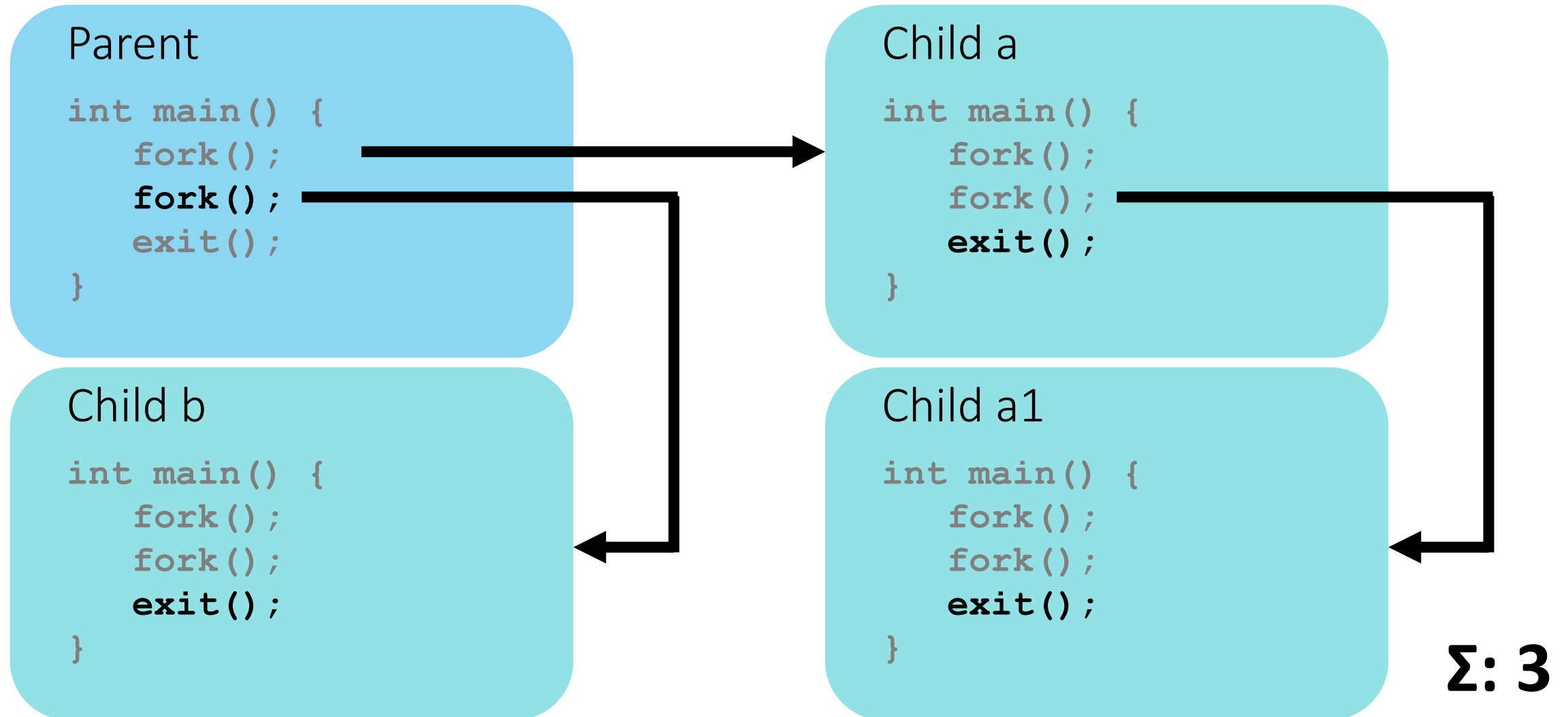
Wie viele Prozesse werden während der Ausführung dieses Programms neu erstellt?

```
int main() {  
    fork();  
    fork();  
    exit();  
}
```

`pid_t fork(void)`: Erstellt einen neuen Prozess, indem der aktuelle Prozess dupliziert wird



Frage 1: Wie viele Prozesse werden während der Ausführung dieses Programms neu erstellt?



Frage 2:

In einem nicht-präemptiven System warten 4 Prozesse mit den unten gegebenen Laufzeiten auf ihre Ausführung.

Welcher Scheduling-Algorithmus sollte genutzt werden, um die durchschnittliche Antwortzeit des Systems möglichst gering zu halten?

A – Laufzeit 9

B – Laufzeit 6

C – Laufzeit 3

D – Laufzeit 5

Gegebene Prozesse: A (9) – B (6) – C (3) – D (5)

FCFS: Weist Prozesse der CPU entsprechend der Ankunftszeit zu – nicht präemptiv

A (9) – B (6) – C (3) – D (5)

Durchschnittliche Antwortzeit: $(9 + (9+6) + (9+6+3) + (9+6+3+5)) / 4 = 16,25$

SJF: Prozess mit kürzester Laufzeit kriegt die CPU zuerst – nicht präemptiv

C (3) – D (5) – B (6) – A (9)

Durchschnittliche Antwortzeit: $(3 + (3+5) + (3+5+6) + (3+5+6+9)) / 4 = 12$

Gegebene Prozesse: A (9) – B (6) – C (3) – D (5)

SRT: Modifizierte Version von SJF, die präemptiv ist: Prozesse werden nach noch verbleibender Laufzeit ausgewählt

Round Robin: Jeder Prozess hat ein Zeitslot (Quantum), nach Verbrauch dieses Quantums wird der Prozess pausiert und wieder in die Warteschlange eingehängt und der nächste Prozess aus der Warteschlange ausgewählt (präemptiv)

Beispiel mit Quantum 3:

A (9) – B (6) – C (3) – D (5) – A (6) – B (3) – D (2) – A (3) – A (3)

Frage 2: Welche Scheduling-Strategie sollte gewählt werden?

Antwort: Shortest Job First (SJF)

Ausführungsreihenfolge:

C (3) – D (5) – B (6) – A (9)

durchschnittliche Antwortzeit/Latenz:

$$(3 + (3+5) + (3+5+6) + (3+5+6+9)) / 4 = 12$$

Frage 3:

Angenommen wir haben nun ein präemptives System, das Shortest Job First (SJF) benutzt. An Zeiteinheit $t=10$ kommen Prozesse E und F dazu. **Was ist der aktuelle Status der Prozesse?**

Ausführungsreihenfolge bei SJF:

C (3) – D (5) – B (6) – A (9)

A – Laufzeit 9 arriving at $t=10$:

B – Laufzeit 6 4 E – Laufzeit 2

~~C – Laufzeit 3~~ F – Laufzeit 7

~~D – Laufzeit 5~~

Frage 3:

Angenommen wir haben nun ein präemptives System, das Shortest Job First (SJF) benutzt. An Zeiteinheit $t=10$ kommen Prozesse E und F dazu. Welchen Algorithmus sollten wir ab jetzt nutzen, um die durchschnittliche Antwortzeit zu minimieren?

A	-	Laufzeit	9	arriving at t=10:	
B	-	Laufzeit	6	4	E - Laufzeit 2
C	-	Laufzeit	3		F - Laufzeit 7
D	-	Laufzeit	5		

Frage 3: Welche Scheduling-Strategie sollte ab $t=10$ gewählt werden?

Antwort: Shortest Remaining Time (SRT)

Ausführungsreihenfolge:

E (2) – B (4) – F (7) – A (9)

durchschnittliche Antwortzeit/Latenz ab $t = 10$:

$$(2+(2+4)+(2+4+7)+(2+4+7+9))/4 = 10,75$$

Frage 4:

Warum gibt es neben den Prozesszuständen **ready** und **running** den Zustand **blocked**? Was kann man mit diesem Zustand erreichen?

Prozesse, die auf ein I/O Ereignis warten (wie z.B. Maus, Tastatur, Netzwerk,...) werden als blocked markiert, damit der Scheduler sie nicht auswählt und dadurch keine CPU-Zeit verschwendet

Warum sind so viele Prozesse im blocked/sleeping Zustand?

Weil viele Prozesse, wie z.B. Browser, Texteditoren, etc. auf einen Input vom User warten.

Aufgabe 1

Wie viele Prozesse werden im Laufe der Ausführung erzeugt? Was würde passieren, wenn die Schleife nicht bei 5 aufhören würde?

1. Iteration: 1 neuer Prozess, insgesamt: 2
 2. Iteration: 2 neue Prozesse, insgesamt: 4
 3. Iteration: 4 neue Prozesse, insgesamt: 8
 4. Iteration: 8 neue Prozesse, insgesamt: 16
 5. Iteration: 16 neue Prozesse, insgesamt: 32
- Insgesamt werden also 31 Prozesse neu erzeugt

Bei unbegrenztem Loop würden wir unbegrenzt viele Prozesse erzeugen ("fork bomb"), welches das System zum Abstürzen bringen kann.

(Mehr Informationen: <https://de.wikipedia.org/wiki/Forkbomb>)

Aufgabe 2

Wie viele Threads werden insgesamt erzeugt? Warum sind es weniger als bei der fork-Version?

Wir sehen, dass wir nur 5 Threads benutzen.

Das liegt daran, dass Threads, im Gegensatz zu Prozessen, Adressräume teilen; jeder Thread hat also Zugriff auf die gleiche `global_variable`, während bei der `fork`-Version jeder Prozess eine eigene `global_variable` hat.

Jeder Thread bearbeitet also die gleiche Variable, so dass nur 5 Threads insgesamt erzeugt werden (*Wegen unzureichender Synchronisation können manchmal auch mehr Threads erzeugt werden, mehr dazu später in der Vorlesung*).

Aufgabe 3

Welche Unterschiede gibt es zwischen der fork- und der Thread-Version? Warum treten diese auf?

Der Mainthread printet die Completion-Message nie, während die fork-Version dies schon tut.

Durch den **exec**-Call wird der Adressraum durch den Adressraum von **ls** ersetzt. Da beide Threads den gleichen Adressraum teilen, wird der ganze Adressraum der beiden Threads durch das **ls**-Programm ersetzt und das ursprüngliche Programm nicht weiter ausgeführt. Bei der fork-Version wird durch die getrennten Adressräume nur der Adressraum vom Kindprozess überschrieben.

Weitere Infos dazu aus der Beschreibung von syscall (*execve()* wird von *execl()* aufgerufen):
All threads other than the calling thread are destroyed during an execve(). Mutexes, condition variables, and other pthreads objects are not preserved.